

Revisiting Polynomial NTRU for FHE: Amortized Bootstrapping with Sparse Keys

Wun-Ting Lin and Ja-Ling Wu

National Taiwan University, Taipei, Taiwan
d07922020@ntu.edu.tw, wjl@cmlab.csie.ntu.edu.tw

Abstract. Fully homomorphic encryption (FHE) enables computation over encrypted data, playing a fundamental role in privacy-preserving machine learning. Recent work has demonstrated that FHE schemes can be constructed under the NTRU assumption, leveraging the benefits of compact ciphertexts. However, existing NTRU-based FHE constructions rely on matrix representations, which break the polynomial ring structure and prevent the direct adoption of modern amortized bootstrapping techniques.

In this work, we revisit NTRU-based FHE by reformulating the matrix-based construction into a standard polynomial-ring setting. We show that the NTRU decryption operation can be decomposed into a set of inner products compatible with FHEW-style accumulators, while preserving the polynomial structure required for further optimization. Building on this formulation, we adapt a recent amortized bootstrapping approach based on monomial-by-polynomial multiplication to the NTRU setting using sparse secret keys.

The resulting scheme combines compact ciphertexts with efficient amortized bootstrapping, reducing both computational cost and bootstrapping key size when the secret key has low Hamming weight. A proof-of-concept Python implementation validates the correctness of the proposed scheme and confirms the expected reduction in bootstrapping cost under conservative security parameters.

Keywords: Fully Homomorphic Encryption · NTRU · Amortized Bootstrapping · Sparse Keys

1 Introduction

Deep learning has become one of the fastest-growing fields of research, with numerous practical applications depending on cloud-based infrastructures [15,14]. This trend has increased the concerns over data confidentiality, motivating extensive research on privacy-preserving machine learning [12,10]. Fully homomorphic encryption (FHE) offers a robust solution by enabling arbitrary computations on encrypted data; however, significant computational overhead, particularly during the bootstrapping phase, restricts its widespread adoption.

The FHE scheme based on the NTRU hardness assumption [9] is recognized for its efficiency. Originating from LTV [11] and BLLN [3] in 2012 and 2013,

NTRU FHE was soon found to be vulnerable to subfield lattice attacks when employing overstretched parameters. However, following the comprehensive analysis by Ducas and van Woerden [6], Bonte et al. [2] have successfully demonstrated a secure NTRU FHE by choosing parameters that lie outside of the overstretched range. With the GSW-like bootstrapping, NTRU FHE achieves competitive bootstrapping performance, supporting nonlinear functions while requiring less memory due to its compact ciphertext structure.

Amortized bootstrapping, first proposed by Micciancio and Sorrell [13] in 2018, allows bitwise FHE schemes, like FHEW [7] and TFHE [5], to process data in parallel; however, these approaches suffer from the complicated implementation and computational overhead. A recent work by Guimarães and Pereira [8] in 2025 introduces a novel amortized bootstrapping method that requires only $\mathcal{O}(h)$ operations per message, where h denotes the Hamming weight of the secret key. This advancement enhances the competitive potential of bitwise schemes as a solution for privacy-preserving machine learning (PPML).

A natural question arises: is it possible to construct an amortized bootstrapping approach under the NTRU hardness assumption that simultaneously exploits data parallelism and compact ciphertext representations? This question motivates our work. We propose a polynomial-based NTRU FHE scheme that supports amortized bootstrapping, requiring only $\mathcal{O}(h)$ operations per message. The primary prior work, FINAL [2], adopts a matrix-NTRU variant to formulate decryption as an inner product, thereby utilizing the accumulator designed by FHEW [7]. However, the lack of polynomial structure and support for automorphisms prevents matrix-NTRU from adopting recent amortized bootstrapping techniques.

In this work, we first present a polynomial-NTRU-based FHE scheme by decoupling polynomial multiplication of the decryption into a sequence of carefully designed inner products to fit the structure used in FHEW-style bootstrapping. Leveraging the polynomial format, we then redesign the amortized bootstrapping mechanism proposed by Guimarães and Pereira for the NTRU setting with ternary secret keys. For a fixed ring dimension N , the external product between RLWE ciphertexts requires $4 \cdot l$ operations due to their tuple representation, whereas the corresponding operation for NTRU ciphertexts needs only l operations. As a consequence, FINAL bootstrapping incurs the cost of $N \cdot l$ operations per message, while the amortized scheme of Guimarães and Pereira achieves a cost of $4 \cdot h \cdot l$, with Hamming weight h . Inheriting the advantages of both approaches, our construction achieves an amortized bootstrapping cost of $h \cdot l$ per message. Moreover, a proof-of-concept Python implementation is provided to demonstrate the feasibility of the proposed polynomial-NTRU-based FHE scheme.

1.1 Our Techniques

Polynomial NTRU Fully Homomorphic Encryption Scheme. We introduce a polynomial-based NTRU Fully Homomorphic Encryption (FHE) scheme,

which differs from the matrix variant in the prior work [2]. This scheme encrypts a vector of n messages, a_i , where $0 \leq i < n$, into a single polynomial $a(X) = \sum_{i=0}^{n-1} a_i \cdot X^i$. By utilizing the Hadamard product of ciphertexts, our scheme produces n parallel outputs from a single NAND gate operation. During the bootstrapping stage, the proposed scheme converts the polynomial multiplication required for NTRU decryption into N inner product operators by independently expressing the computation with respect to each output index k .

Given two polynomials $a(X) = \sum_{i=0}^{n-1} a_i \cdot X^i$ and $f(X) = \sum_{i=0}^{n-1} f_i \cdot X^i$, the multiplication between $a(X)$ and $f(X)$ can be written as $c(X) = a(X) * f(X) = \sum_{i=0}^{n-1} c_i \cdot X^i$ as each c_k is defined as:

$$c_k = \sum_{i=0}^k a_{k-i} \cdot f_i - \sum_{i=k+1}^n a_{n+k-i} \cdot f_i \quad (1)$$

It's clear that for each k , the coefficient c_k can be computed as the inner product between $a(X^{-1}) \cdot X^k$ and $f(X)$, that is $c_k = \langle a(X^{-1}) \cdot X^k, f(X) \rangle$. The formulation of c_k is compatible with GSW-like accumulators. By incorporating a repacking technique, the proposed scheme can be bootstrapped using the FHEW-based accumulator while preserving the original NTRU polynomial structure.

Amortized Bootstrapping with Sparse Secure Key. The motivation for reverting the matrix-based construction to a polynomial structure is to enable the adoption of recent amortized bootstrapping techniques proposed by Guimarães and Pereira [8], which fundamentally rely on polynomial representations. In this work, we extend the amortized bootstrapping algorithm from Ring Learning with Errors (RLWE) to the NTRU polynomial setting, where both operations are defined over the ring $\mathcal{R} := \mathbb{Z}[X]/\langle X^n + 1 \rangle$

Let $a(X) \in \mathcal{R}$ be a ciphertext polynomial and let $f(X) \in \mathcal{R}$ be a secret key polynomial with Hamming weight h . The NTRU decryption operation involves a polynomial multiplication, denoted as $*$ between $a(X)$ and $f(X)$, which can be expressed as follows:

$$\begin{aligned} a(X) * f(X) &= a(X) \cdot \sum_{i \in \text{idx}(f)} f_i X^i = \sum_{j \in \text{idx}(f)} \left(\sum_{i=0}^N a_i f_j X^i \right) \odot X^j \\ &= \sum_{j \in \text{idx}(f)} \text{MPMul}(a(X) \cdot f_j, \text{Enc}(X^j)) \end{aligned} \quad (2)$$

Here, $\text{idx}(f)$ denotes the set of indices corresponding to the non-zero coefficients of $f(X)$, and $\odot$ represents the monomial-by-polynomial multiplication introduced by Guimarães and Pereira, also referred to as **MPMul**. Based on Equation 2, the polynomial multiplication, which would typically require N inner products, can be reduced to h monomial-by-polynomial multiplication. Consequently, the cost of our amortized bootstrapping scheme is $\mathcal{O}(h)$. An overview of the proposed scheme is illustrated in Figure 1.

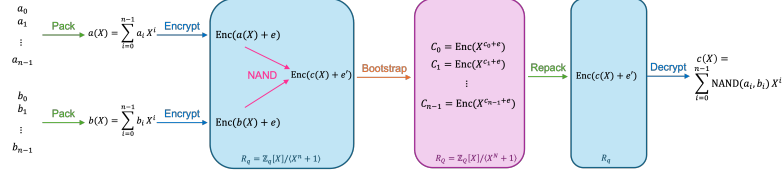


Fig. 1. The overview of our NTRU-based amortized bootstrapping scheme.

Proof-of-Concept Implementation. We provide an open-source Python implementation¹ to demonstrate the feasibility of the proposed scheme. Due to the lack of full optimization, our goal is not to directly compare wall-clock performance with existing state-of-the-art implementations. Instead, we validate the correctness and asymptotic behavior of both native bootstrapping, which requires N inner products, and amortized bootstrapping, which relies on only h monomial-by-polynomial multiplications (MPMuls). From a theoretical perspective, amortized bootstrapping reduces the asymptotic operation cost and achieves an algorithmic speedup of up to $1.8\times$ over native bootstrapping under the NTRU setting. In addition, the size of the bootstrapping key is reduced from $\mathcal{O}(n)$ to $\mathcal{O}(h \cdot \lceil \log_2 n \rceil)$, resulting in significant space savings when $h \ll n$.

Limitation. At the early stage of development, the proposed scheme does not yet have a fully optimized implementation in low-level languages such as C or Rust. Instead, we provide a proof-of-concept Python implementation to validate correctness and feasibility. Our construction employs four auxiliary keys in the amortized bootstrapping approach. Although this introduces additional keys, the overall key size remains smaller than that of standard GSW-like FHE schemes. In contrast to standard NTRU-based schemes that support integers in the range $[-\frac{p}{2}, \frac{p}{2})$, where p denotes the plaintext modulus, the current implementation supports only binary polynomials with coefficients in $\{0, 1\}$. Extending the proposed scheme to support larger plaintext spaces is feasible within the NTRU framework and is left as future work.

1.2 Contribution

1. **Polynomial-NTRU FHE construction.** We present a fully homomorphic encryption scheme based on the NTRU hardness assumption that operates entirely over polynomial rings, enabling native support for ring-based operations and automorphisms.
2. **Accumulator-compatible decryption reformulation.** We introduce a repacking technique that reformulates NTRU decryption from polynomial multiplication into a collection of N inner products indexed by output coefficients, making our ciphertexts compatible with FHEW-style accumulators.

¹ Github: TO BE PUBLISHED WITH THE PAPER.

3. **Amortized bootstrapping via monomial-by-polynomial multiplication.** Building on the above construction, we design an amortized bootstrapping scheme for NTRU that exploits monomial-by-polynomial multiplication (MPMu1). For secret keys with Hamming weight h , our approach reduces the amortized bootstrapping cost to $\mathcal{O}(h)$.
4. **Bootstrapping key size reduction with sparse secret.** By explicitly recording the non-zero positions of sparse secret keys, we can reduce the bootstrapping key size from $\mathcal{O}(n)$ to $\mathcal{O}(h \log_2 n)$, resulting in significant memory savings when $h \ll n$.

2 Preliminaries

2.1 Notation

For the clarity of the definitions of notations used in this writeup, we redefine the involved notations in this subsection. Let $\mathcal{R} = \mathbb{Z}[X]/\langle X^n + 1 \rangle$ be the $2n$ -th cyclotomic ring, where $n = 2^k$ with some $k \in \mathbb{N}$. The ring $\mathcal{R}_q = \mathcal{R}/q\mathcal{R} \equiv \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$ represents the quotient ring modulo q . Vectors are denoted by bold lowercase letters (e.g., $\mathbf{a}, \mathbf{b} \in \mathbb{Z}^n$), and polynomials are denoted by italic lowercase letters (e.g., $a, b \in \mathcal{R}$).

Any element a of $\mathcal{R}_q$ can be expressed as a unique polynomial of degree smaller than n . Specifically, write $a = \sum_{i=0}^{n-1} a_i \cdot X^i$, where the coefficients $a_0, a_1, \dots, a_{n-1}$ are elements of $\mathbb{Z}_q$. The coefficient vector of a polynomial a is defined as $\mathbf{a} = \phi(a) := (a_0, \dots, a_{n-1})^T \in \mathbb{Z}_q^n$, and the infinity norm of $\mathbf{a}$ is given by $\|\mathbf{a}\| := \|\phi(a)\|$, which represents the standard L^∞ -norm of the vector.

We further define the Hadamard product $\mathbf{a} \circ \mathbf{b}$ as the component-wise multiplication of the vectors $\mathbf{a}$ and $\mathbf{b}$. Let $\mathbf{a} = \phi(a) = (a_0, \dots, a_{n-1})^T$ and $\mathbf{b} = \phi(b) = (b_0, \dots, b_{n-1})^T$, the Hadamard product is given by:

$$a \circ b = \phi(a) \circ \phi(b) = \mathbf{a} \circ \mathbf{b} = (a_0 \cdot b_0, \dots, a_{n-1} \cdot b_{n-1})^T$$

We denote the component-wise square of a polynomial as $a^{\circ 2} := a \circ a$. For polynomials a and b with binary coefficients, the squared difference satisfies $(a - b)^{\circ 2} = a + b - 2(a \circ b)$.

Next, we define the set $\mathbb{M} := \{\pm b \cdot X^k : b \in \{0, 1\}, k \in \mathbb{N}\}$, which contains monomials of degree k with ternary coefficients. Last, $\langle a, b \rangle$ stands for the inner product between two polynomials a and b , and $a * b$ represents the polynomial multiplication of a and b .

2.2 Discrete Gaussian Distribution

A discrete Gaussian distribution over $\mathbb{Z}$ is a probability distribution where each integer is sampled with a probability proportional to its probability mass function value under a Gaussian distribution over $\mathbb{R}$. Specifically, for $\sigma, c \in \mathbb{R}$ with $\sigma > 0$, we define the Gaussian function as $\rho_{\sigma, c}(x) = \exp(-\frac{|x-c|^2}{2\sigma^2})$. By normalizing this over the integers, we define $\rho_{\sigma, c}(\mathbb{Z}) = \sum_{i=-\infty}^{\infty} \rho_{\sigma, c}(i)$. The probability of sampling $x \in \mathbb{Z}$ is thus given by $\rho_{\sigma, c}(x)/\rho_{\sigma, c}(\mathbb{Z})$. If the center c is zero, we simplify the notation to χ_σ .

2.3 Gadget Decompositions

Let $l := \lceil \log_B q \rceil$ and define $\mathbf{g} := (B^0, B^1, \dots, B^{l-1})$. For any $k \in \mathbb{Z}_q$, represent k by its centered integer representative in the interval $[-\frac{q}{2}, \frac{q}{2})$. We then define its base- B decomposition as $\mathbf{g}^{-1}(k) = (k_0, \dots, k_{l-1})$ where each coefficient satisfies $|k_i| \leq \frac{B}{2}$ for all $i \in [l]$. We have that $\mathbf{g}^{-1}(k) \cdot \mathbf{g} = k$. For any polynomial $a \in \mathcal{R}_q$ with coefficients $(a_0, \dots, a_{n-1})$, we define $\mathbf{g}^{-1}(a) := \sum_{i=0}^{n-1} \mathbf{g}^{-1}(a_i) \cdot X^i$. Thus, it is clear that $\mathbf{g}^{-1}(a) \cdot \mathbf{g} = \sum_{i=0}^{n-1} \mathbf{g}^{-1}(a_i) \cdot \mathbf{g} \cdot X^i = \sum_{i=0}^{n-1} a_i \cdot X^i = a$.

2.4 NTRU Problem

For integers $n > 0, q > 1$ and a real parameter $\sigma > 0$, let $\mathcal{R}_q := \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$. Given $f, g \leftarrow \chi_\sigma^n$ where f is invertible in $\mathcal{R}_q$, we define the following problems:

- (n, q, σ)-Computational NTRU:** Recover f and g from the public key $h := g \cdot f^{-1} \pmod{q}$.
- (n, q, σ)-Decisional NTRU:** Distinguish h from a polynomial u sampled uniformly at random from $\mathcal{R}_q$.

3 Polynomial NTRU FHE Scheme

The fundamental construction of NTRU was introduced by Hoffstein, Pipher, and Silverman in 1998 [9]. In this work, we adopt an anti-circulant formulation of NTRU over the ring $\mathbb{Z}_q[X]/\langle X^n + 1 \rangle$, which is a common choice in modern lattice-based cryptography, as reflected in recent works such as [2].

Following the exhaustive study of the overstretched NTRU problem by Ducas and van Woerden [6], Bonte et al. proposed FINAL [2], demonstrating the feasibility of constructing a fully homomorphic encryption scheme under the NTRU assumption. FINAL employs a matrix-NTRU variant that enables decryption to be simulated as an inner product between a ciphertext encoding a single bit and the first column of the secret key matrix, denoted by $\text{col}_0(\mathbf{F}^{-1})$.

While this matrix-based formulation facilitates accumulator-based bootstrapping, it departs from the standard polynomial ring structure and therefore does not support ring automorphisms. This structural limitation prevents the direct adoption of amortized bootstrapping techniques that rely on polynomial representations. To address this limitation, we develop a new NTRU-based FHE scheme that operates entirely in the polynomial ring setting.

3.1 Overview

As with most FHEW-type frameworks, our approach introduces two encrypted schemes: a base scheme and an accumulator scheme. The base scheme supports NAND gate evaluation and enables encrypted data manipulation, whereas the accumulator scheme provides external product functionality and is specifically designed for the accumulator used in FHEW-type bootstrapping.

In the proposed method, plaintext data are represented as polynomials and encrypted in batches under the base scheme, namely $\text{Enc}(m = \sum_{i=0}^{n-1} m_i X^i)$. In contrast, the accumulator scheme encrypts a single integer in the exponent as a monomial, i.e., $\text{Enc}(X^u)$. This representation is employed to support homomorphic rotation, also known as blind rotation, as described in prior work [5]. In the following section, we present the detailed design of both the base scheme and the accumulator scheme, each constructed under the NTRU hardness assumption.

3.2 NTRU-based FHE Scheme

Let $R_q := \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$ be the cyclotomic ring. The coefficient vector of polynomial a is denoted by $\phi(a) = (a_0, a_1, \dots, a_{n-1})^T$. The NTRU-based Fully Homomorphic Encryption (FHE) scheme is defined as follows:

- $\text{NTRU.ParamGen}(1^\lambda)$: Output the parameters (n, q, σ) based on the security parameter λ .
- $\text{NTRU.KeyGen}(n, q, \sigma)$: Sample $f' \leftarrow \chi_\sigma^n$ and set the secret key $\text{sk} := f = 1 + 4f'$ until f is invertible in R_q . Define the public evaluation key $\text{evk} := (g + \lfloor \frac{5q}{8} \rfloor) \cdot f^{-1}$, where $g \leftarrow \chi_\sigma^n$ and output (evk, sk) .
- $\text{NTRU.Enc}(m, \text{sk})$: Given a binary polynomial $m = \sum_{i=0}^{n-1} m_i X^i \in R_q$, where $m_i \in \{0, 1\}$. Let $\Delta = \lfloor q/4 \rfloor$ and sample $g \leftarrow \chi_\sigma^n$. Output the ciphertext as:

$$c = (g + \Delta \cdot m) \cdot f^{-1} \in R_q$$

- $\text{NTRU.Dec}(c, \text{sk})$: Compute $r = c \cdot f \pmod{q}$ and output the decrypted message $m' = \lfloor \frac{2r}{q} \rfloor \pmod{2}$.

Lemma 1 (Correctness of Decryption). *Let $c := (g + \Delta \cdot m) \cdot f^{-1} \in R_q$ be an encryption of a binary polynomial $m = \sum_{i=0}^{n-1} m_i X^i$ with $m_i \in \{0, 1\}$, where $\Delta = \lfloor q/2 \rfloor$. Considering a ternary g and setting q sufficiently large, then $\text{NTRU.Dec}(c, \text{sk})$ outputs $m' = m$.*

The proof of Lemma 1 is deferred to Appendix A. Following the design principles of FINAL [2], a fresh ciphertext encrypts a message $m \in \mathbb{Z}_4$ using the scaling factor $\Delta := \lfloor q/4 \rfloor$. Subsequently, the output of the NAND gate yields a message in $\mathbb{Z}_2$, which is then scaled by $\lfloor q/2 \rfloor$.

- $\text{NTRU.Nand}(c_0, c_1, \text{evk})$: Let $y := \text{NAND}(a, b) = 1 - a \circ b$ be the target polynomial, with coefficients $y_i = \text{NAND}(a_i, b_i) = 1 - a_i \cdot b_i$. Given evaluation key evk and $c_0 = \text{Enc}(a) = (g_0 + \lfloor q/4 \rfloor \cdot a) \cdot f^{-1}$ and $c_1 = \text{Enc}(b) = (g_1 + \lfloor q/4 \rfloor \cdot b) \cdot f^{-1}$ where a, b are binary polynomials. The output ciphertext c_{NAND} , representing $\text{Enc}(y)$, is computed as:

$$c_{\text{NAND}} := \text{evk} - c_0 - c_1$$

Lemma 2 (Correctness of NTRU.Nand). *Let $c_0 := (g_0 + \lfloor q/4 \rfloor \cdot a) \cdot f^{-1}$ and $c_1 := (g_1 + \lfloor q/4 \rfloor \cdot b) \cdot f^{-1}$ be fresh encryptions of binary polynomials a and b . Let evk be an evaluation key generated with the ternary polynomial g . For $c = \text{NTRU.Nand}(c_0, c_1, \text{evk})$, if for all i , $|\phi(g_0)_i + \phi(g_1)_i| < \frac{q-20}{8}$, then $\text{NTRU.Dec}(c, \text{sk})$ outputs $y = \text{NAND}(a, b) = 1 - a \circ b$.*

The proof of Lemma 2 is deferred to Appendix B.

3.3 NTRU-based Accumulator Scheme

Building on our accumulator scheme, we introduce two ciphertext formats that support external products, enabling the use of an accumulator. Similar to the NGS scheme in FINAL[2], the first format encrypts a ternary polynomial in $\mathcal{R}_Q$ ($Q \gg q$), whereas the second format represents the ciphertext as a vector over $\mathcal{R}_Q$ via gadget decomposition.

- **ACC.ParamGen**(1^λ): Given security parameter λ , generate (N, Q, ς, B, l) , where B is the base value of gadget decomposition while $l := \lceil \log_B Q \rceil$.
- **ACC.KeyGen**(N, Q, ς): Sample $f'_Q \leftarrow \chi_\varsigma^N$ and set the secret key $\mathbf{sk} := f_Q = 1 + 4 \cdot f'_Q$ until f_Q is invertible in $\mathcal{R}_Q$, then output $\mathbf{sk}$.
- **ACC.EncS**($u, \mathbf{sk}$): Sample $g \leftarrow \chi_\varsigma^N$ and define $\Delta := \lfloor Q/4 \rfloor$. Output a scalar ciphertext $c = g/f_Q + \Delta \cdot u \in \mathcal{R}_Q$, which is a scalar encryption of a ternary polynomial $u \in \mathcal{R}_Q = \mathbb{Z}_Q[X]/\langle X^N + 1 \rangle$.
- **ACC.EncV**($v, \mathbf{sk}$): Define the gadget vector $\mathbf{g} = (B^0, B^1, \dots, B^{l-1})^T$ and sample $\mathbf{g} = (g_0, \dots, g_{l-1})$ with $g_i \leftarrow \chi_\varsigma^N$. Output a vector encryption $\mathbf{c} = \mathbf{g}/f_Q + \mathbf{g} \cdot v \in \mathcal{R}_Q^l$, which encrypts a decomposed form of message $v \in \mathbb{M} = \{\pm b \cdot X^k : b \in \{0, 1\}, k \in \mathbb{N}\}$.

External Product. The external product of a scalar ciphertext c and a vector ciphertext $\mathbf{c}$ is defined as follows:

$$c_{\text{mult}} = c \boxtimes \mathbf{c} := \mathbf{g}^{-1}(c) \cdot \mathbf{c} \in \mathcal{R}_Q$$

Since $\mathbf{g}^{-1}(c) \cdot \mathbf{g} = c$, the external product c_{mult} yields a valid scalar encryption of the product $u \cdot v$ with g_{mult} is sufficiently small. The detailed algebraic expansion and correctness proof are provided in Appendix C. To maintain correctness, we require $\|u \cdot v\| < 4$; otherwise, the scaling by Δ may introduce significant round-off error, resulting in an ill-formed ciphertext. Given that $v \in \mathbb{M}$ and u is a ternary polynomial, we have $\|u \cdot v\| \leq \|u\| < 2$, which satisfies this condition.

4 Efficient Amortized Bootstrapping

In this section, we present a detailed description of the bootstrapping procedures for the NTRU-based schemes, including a native bootstrapping procedure that relies on a ternary **CMux** and an amortized bootstrapping procedure that leverages monomial-by-polynomial multiplication (**MPMul**).

4.1 Native bootstrapping

Ternary CMux. A ternary **CMux** _{i} is introduced for the i -th coefficient of the secret key f , where $f_i \in \{-1, 0, 1\}$. First, we define two bootstrapping keys, denoted by $\mathbf{bsk}_{i,0}$ and $\mathbf{bsk}_{i,1}$, as follows:

$$\begin{cases} f_i = -1 : & \mathbf{bsk}_{i,0} := \text{ACC.EncV}(0), & \mathbf{bsk}_{i,1} := \text{ACC.EncV}(1), \\ f_i = 0 : & \mathbf{bsk}_{i,0} := \text{ACC.EncV}(0), & \mathbf{bsk}_{i,1} := \text{ACC.EncV}(0), \\ f_i = 1 : & \mathbf{bsk}_{i,0} := \text{ACC.EncV}(1), & \mathbf{bsk}_{i,1} := \text{ACC.EncV}(0). \end{cases}$$

Then the ternary $\text{CMux}_i(c_i)$ for an input a_i is computed as:

$$\text{CMux}_i(a_i) = \mathbf{1} + (X^{a_i} - 1) \cdot \text{bsk}_{i,0} + (X^{-a_i} - 1) \cdot \text{bsk}_{i,1}$$

In the above expression, the operations $X^{a_i} \cdot \text{bsk}_{i,0}$ and $X^{-a_i} \cdot \text{bsk}_{i,1}$ correspond to ring rotations applied to the bootstrapping keys. These rotations are implemented as index permutations and therefore incur no homomorphic multiplication and no additional noise growth. The term $\mathbf{1}$ denotes a noiseless encryption of one, which is instantiated as the gadget vector $\mathbf{g}$. It is straightforward to verify that $\text{CMux}_i(a_i) = \text{ACC}.\text{EncV}(X^{a_i \cdot f_i})$

Inner Product. Given a cleartext vector $\mathbf{a} = (a_0, \dots, a_{n-1})^T$ and an encrypted vector $\mathbf{f} = (f_0, \dots, f_{n-1})^T$, the inner product $\langle \mathbf{a}, \mathbf{f} \rangle = \sum_{i=0}^{n-1} a_i f_i$ can be accumulated with the ternary $\text{CMux}_i(a_i)$ over all indices i .

Starting from a scalar encryption of $\mathbf{1}$, the inner product is obtained as

$$\begin{aligned} \text{ACC}.\text{EncS}\left(X^{\langle \mathbf{a}, \mathbf{f} \rangle}\right) &= \text{ACC}.\text{EncS}(\mathbf{1}) \boxtimes \text{CMux}_0(a_0) \cdots \boxtimes \text{CMux}_{n-1}(a_{n-1}) \\ &= \text{ACC}.\text{EncS}(\mathbf{1}) \boxtimes \text{ACC}.\text{EncV}(X^{a_0 \cdot f_0}) \cdots \boxtimes \text{ACC}.\text{EncV}(X^{a_{n-1} \cdot f_{n-1}}) \\ &= \text{ACC}.\text{EncS}\left(X^{\sum_{i=0}^{n-1} a_i \cdot f_i}\right). \end{aligned} \quad (3)$$

Here, the operator $\boxtimes$ denotes the external product between the scalar encryption $\text{ACC}.\text{EncS}$ and vector encryption $\text{ACC}.\text{EncV}$. As a result, the output of Equation 3 encrypts $X^{\langle \mathbf{a}, \mathbf{f} \rangle}$ under the $\text{ACC}.\text{EncS}$ format.

Polynomial Multiplication We further extend the proposed construction to support polynomial multiplication. Let $a(X) = \sum_{i=0}^{n-1} a_i X^i$ and $f(X) = \sum_{i=0}^{n-1} f_i X^i$, where $a(X)$ is given in the clear and $f(X)$ is encrypted, with coefficients $f_i \in \{-1, 0, 1\}$. Their polynomial multiplication $c(X) = a(X) * f(X) = \sum_{k=0}^{n-1} c_k X^k$ is defined via negacyclic convolution, where each coefficient c_k satisfies the Equation 1.

To express Equation 1 as an inner product, we define a vector $\tilde{\mathbf{a}}_k := \phi(\tilde{a}_k(X)) := [a_k, a_{k-1}, \dots, a_0, -a_{n-1}, \dots, -a_{k+1}]$ and $\mathbf{f} := \phi(f(X))$. With this notation, the coefficient c_k can be written compactly as $c_k = \langle \tilde{\mathbf{a}}_k, \mathbf{f} \rangle$. Observe that the polynomial $\tilde{a}_k(X)$ satisfies $\tilde{a}_k(X) = a(X^{-1}) \cdot X^k$. Therefore, each coefficient c_k of the polynomial multiplication can be computed as $c_k = \langle \phi(a(X^{-1}) \cdot X^k), \phi(f(X)) \rangle$.

The homomorphically accumulated polynomial derived from the ciphertext a is denoted by ACC . The k -th coefficient of ACC is then computed as

$$\begin{aligned} \text{ACC}_k &= \text{BS}.\text{EncS}\left(X^{\langle \tilde{\mathbf{a}}_k, \mathbf{f} \rangle}\right) \\ &= \text{BS}.\text{EncS}\left(X^{\langle \phi(a(X^{-1}) \cdot X^k), \phi(f(X)) \rangle}\right) \\ &= \text{BS}.\text{EncS}(\mathbf{1}) \boxtimes \text{CMux}_0(a_k) \boxtimes \text{CMux}_1(a_{k-1}) \cdots \boxtimes \text{CMux}_{n-1}(-a_{k+1}) \end{aligned}$$

Algorithm 1: Native Bootstrapping

Input: $a \in \mathcal{R}_q^n$, $\{\text{bsk}_{i,j}\}$, ksk^{aut}

```

1  $\phi(a) \leftarrow \lfloor 2N\phi(a)/q \rfloor$ ;  $\tilde{a} \leftarrow \phi(\text{Automorphism}(a, -1))$ ;
2 for  $0 \leq k < n$  do
3    $\mathbf{c} \leftarrow \text{rotate}(\tilde{a}, k)$ ;  $\text{ACC}_k \leftarrow \lfloor \frac{Q}{8} \rfloor X^{N/2} \sum_{i=0}^{N-1} X^i$ ;
4   for  $0 \leq i < n$  do
5      $\text{ACC}_k \leftarrow \text{ACC}_k \boxplus \text{CMux}_i(c_i)$ 
6    $\text{ACC}_k \leftarrow \text{ACC}_k + \lfloor \frac{Q}{8} \rfloor \sum_{i=0}^{N-1} X^i$ ;
7 return  $\text{KeySwitch}(\text{ModSwitch}(\text{ACC}), \text{ksk}^{\text{ext}})$ ;
```

Bootstrapping with Dense Key A test vector is required to extract the message stored in the exponent, similar to standard FHEW-style bootstrapping mechanisms. In the base encryption scheme, a ciphertext encrypts a message $m \in \{0, 1\}$ with the form $c = (g + \Delta \cdot m) \cdot f^{-1}$.

When bootstrapping, the ciphertext c is first scaled by a factor of $\frac{2N}{q}$, and the homomorphic decryption procedure based on the CMux_i yields an accumulator ciphertext denoted by ACC . As a result, the value of the accumulated ciphertext satisfies $\text{ACC} = X^{\frac{2N}{q} \cdot (g + \frac{q}{2} \cdot m)} \approx X^{N \cdot m}$.

Given test vector $T(X) := X^{N/2} \cdot \sum_{i=0}^{N-1} X^i$, multiplying the accumulator gives $\text{ACC} = T(X) \cdot X^{N \cdot m}$. When $m = 0$, the result is $\text{ACC} = T(X)$, whereas for $m = 1$ we obtain $\text{ACC} = T(X) \cdot X^N = -T(X)$ due to negacyclicity. After scaling by $\lfloor q/8 \rfloor$ and adding the polynomial $\lfloor q/8 \rfloor \cdot \sum_{i=0}^{N-1} X^i$ produces

$$\text{ACC} = \begin{cases} 0 & \text{if } m = 0 \\ \lfloor q/4 \rfloor & \text{if } m = 1 \end{cases}$$

The output of the accumulator is stored in the constant term of ACC , which is encrypted under the ACC.EncS representation. We summarize the proposed bootstrapping procedure in Algorithm 1.

4.2 Amortized Bootstrapping

Assume that f is a ternary polynomial. Given a clear polynomial a and an encrypted form of f , the polynomial multiplication $a * f$ can be computed by exploiting the sparsity of f in Equation 2. Since the summation only iterates over $\text{idx}(f)$, the complexity of computing $a * f$ is proportional to the Hamming weight h , requiring exactly h MPMu1 operations.

Monomial-times-polynomial multiplication (MPMu1). Let $u \in R_Q$ be a clear polynomial and $v \in \mathbb{Z}_N$ be an encrypted value represented in exponent as an NTRU vector encryption. Let $\phi(u) = (u_0, u_1, \dots, u_{N-1})^T$ denote the coefficient vector of u , multiplication by the monomial X^v corresponds to a negacyclic rotation of the coefficients $\phi(u)$. To realize this operation homomorphically, we employ a power-of-two decomposition of v .

Algorithm 2: MPMul

Input: $\mathbf{c}, \mathbf{bsk}_k^{\text{rot}}, \mathbf{ksk}^{\text{aut}}$

- 1 **for** $0 \leq i < l$ **do**
- 2 $\forall j < 2^i : \hat{\mathbf{c}}_j \leftarrow (\text{Automorphism}(\mathbf{c}_{N-2^i+j}, -1) - \mathbf{c}_j) \boxplus \mathbf{bsk}_{i,k}^{\text{rot}} + \mathbf{c}_j;$
- 3 $\forall 2^i \leq j < N - 2^i : \hat{\mathbf{c}}_j \leftarrow (\mathbf{c}_{j-2^i} - \mathbf{c}_j) \boxplus \mathbf{bsk}_{i,k}^{\text{rot}} + \mathbf{c}_j;$
- 4 $\mathbf{c} \leftarrow \hat{\mathbf{c}};$
- 5 **return** $(\mathbf{c}_0, \dots, \mathbf{c}_{N-1});$

Let $l = \lceil \log_2 N \rceil$ and $\mathbf{g}^{-1}(v) = (v_0, v_1, \dots, v_{l-1}) \in \{0, 1\}^l$ be the binary decomposition such that $\mathbf{g}^{-1}(v) \cdot \mathbf{g} = v$ with the gadget vector $\mathbf{g} = (2^0, 2^1, \dots, 2^{l-1})$. For each $i \in \{0, \dots, l-1\}$, we homomorphically multiply u by $X^{v_i \cdot 2^i}$, where $v_i \in \{0, 1\}$. Specifically, multiplying by $X^{v_i \cdot 2^i}$ is defined as:

$$f(u_0, \dots, u_{N-1}) = \begin{cases} (u_0, u_1, \dots, u_{N-1}), & \text{if } v_i = 0 \\ (-u_{N-2^i}, \dots, -u_{N-1}, u_0, \dots, u_{N-2^i-1}), & \text{if } v_i = 1 \end{cases}$$

By iteratively applying the rotation for all $i \in \{0, \dots, l-1\}$, we obtain $u \odot \prod_{i=0}^{l-1} X^{v_i \cdot 2^i} = u \odot X^{\sum_{i=0}^{l-1} v_i \cdot 2^i} = u \odot X^v$. The bits v_i are integrated into the scheme via bootstrapping keys, denoted as $\mathbf{bsk}_i^{\text{rot}}$, which are predefined as $\mathbf{bsk}_i^{\text{rot}} := \text{ACC.EncV}(v_i)$. The algorithmic realization of MPMul is detailed in Algorithm 2.

Amortized Polynomial Multiplication. By leveraging the MPMul operation, we can efficiently compute the polynomial multiplication $a * f$ for a sparse polynomial f . Let $f = \sum_{j \in \text{idx}(f)} f_j X^j$ be a polynomial with Hamming weight h , where $N > j_0 > j_1 > \dots > j_{h-1} \geq 0$ denote the indices of the non-zero coefficients that are sorted in descending order. We define a difference vector $\mathbf{d} := \text{diff}(f) \in [0, N]^h$ as:

$$\mathbf{d}[i] = \begin{cases} j_i - j_{i+1} & \text{if } 0 \leq i < h-1, \\ j_i & \text{if } i = h-1. \end{cases}$$

It follows that $\mathbf{d}[i] \geq 0$. The polynomial multiplication $a * f$ is then computed by refactoring the summation into a nested sequence of rotations and additions:

$$\begin{aligned} a * f &= \sum_{i=0}^{h-1} (a \cdot f_{j_i}) \odot X^{j_i} = a \cdot f_{j_0} \odot X^{j_0} + \dots + a \cdot f_{j_{h-1}} \odot X^{j_{h-1}} \\ &= \text{MPMul}(\dots(\text{MPMul}(a \cdot f_{j_0}, X^{\mathbf{d}[0]})\dots + a \cdot f_{j_{h-1}}, X^{\mathbf{d}[h-1]}) \end{aligned}$$

By structuring the computation this way, the scheme requires exactly h MPMul operations for a polynomial multiplication between a and f .

$$\begin{cases} f_j = -1 : & \mathbf{bsk}_j^{\text{sign}} := \text{ACC.EncV}(1), \\ f_j = 1 : & \mathbf{bsk}_j^{\text{sign}} := \text{ACC.EncV}(0). \end{cases} \quad (4)$$

Algorithm 3: Amortized Bootstrapping

Input: $a \in \mathcal{R}_q^n$, $\mathbf{bsk}^{\text{rot}}$, $\mathbf{bsk}^{\text{sign}}$, $\mathbf{ksk}^{\text{aut}}$, $\mathbf{ksk}^{\text{ext}}$

- 1 $\phi(a) \leftarrow \lfloor 2N\phi(a)/q \rfloor$; $\text{ACC}_k \leftarrow \lfloor \frac{Q}{8} \rfloor X^{N/2} \sum_{i=0}^{N-1} X^i$ for $0 \leq k < n$;
- 2 **for** $0 < i < h-1$ **do**
- 3 **for** $0 \leq k < n$ **do**
- 4 $\mathbf{p} \leftarrow \text{rotate}(\text{ACC}_k, X^{a_k})$, $\mathbf{z} \leftarrow \text{rotate}(\text{ACC}_k, X^{-a_k})$;
- 5 $\text{ACC}_k \leftarrow (\mathbf{p} - \mathbf{z}) \boxplus \mathbf{bsk}_k^{\text{sign}} + \mathbf{z}$;
- 6 $\text{ACC} \leftarrow \text{MPMul}(\text{ACC}, \mathbf{bsk}_i^{\text{rot}}, \mathbf{ksk}^{\text{aut}})$;
- 7 $\text{ACC}_k \leftarrow \text{ACC}_k + \lfloor \frac{Q}{8} \rfloor \sum_{i=0}^{N-1} X^i$ for $0 \leq k < n$;
- 8 **return** $\text{KeySwitch}(\text{ModSwitch}(\text{ACC}), \mathbf{ksk}^{\text{ext}})$;

Bootstrapping with Sparse Key. Similar to the native bootstrapping procedure, a test vector is introduced to extract the message from the exponent. By adopting a mechanism analogous to the native bootstrapping in Section 4.1 but replacing core polynomial multiplications with an amortized version based on MPMul , we achieve bootstrapping with h MPMul operations per message.

Since our secret key f is a ternary polynomial (i.e., $f_j \in \{-1, 0, 1\}$), we introduce an additional bootstrapping key, denoted as $\mathbf{bsk}^{\text{sign}}$, to distinguish whether a non-zero coefficient f_j is positive or negative. Let $\text{idx}(f)$ denote the set of indices corresponding to the non-zero coefficients of f . For each $j \in \text{idx}(f)$, the $\mathbf{bsk}^{\text{sign}}$ is defined as Equation 4.

Consequently, the details of the proposed amortized bootstrapping procedure are formally presented in Algorithm 3.

4.3 Modulus Switching

The final stage of bootstrapping converts the output of the accumulator, which encodes the homomorphically decrypted message across n scaled accumulator ciphertexts in the ACC.EncS form, back into the base encryption scheme that represents the message as a single polynomial ciphertext under NTRU.Enc .

Let q denote the modulus of the base scheme, and Q stands for the modulus of the accumulator scheme. Since $q \ll Q$, we perform modulus switching to map a ciphertext $\phi(c)_Q = (c_0, c_1, \dots, c_{N-1})$ under modulus Q to a ciphertext $\phi(c)_q = (c'_0, c'_1, \dots, c'_{N-1})$ under modulus q by scaling each coefficient as follows:

$$c'_i = \left\lfloor c_i \cdot \frac{q}{Q} \right\rfloor, \quad \text{for } 0 \leq i < N.$$

4.4 Key Switching

The proposed work incorporates two key switching procedures. The first is designed to transform the ciphertext encrypted with $f_Q(X^{-1})$ back to f_Q , which is required from MPMul when handling the sign change of coefficients during the rotation. The second procedure facilitates the transition from the ciphertext of

Table 1. Summary of bootstrapping and key switching keys.

Name	Native	Amortized	Purpose / Operation
$\mathbf{bsk}$	✓	–	Ternary CMUX rotation
$\mathbf{bsk}^{\text{rot}}$	–	✓	Sparse monomial position indexing
$\mathbf{bsk}^{\text{sign}}$	–	✓	Handle $\pm c_u$ rotation
$\mathbf{ksk}^{\text{aut}}$	–	✓	Automorphism: $f_Q(X^{-1}) \rightarrow f_Q$
$\mathbf{ksk}^{\text{ext}}$	✓	✓	Switching: $\text{ACC}.\text{EncS} \rightarrow \text{NTRU}.\text{Enc}$

the accumulator scheme with key f_Q to that of the base scheme under key f . To distinguish between them, we refer to the former as *automorphism key switching*, associated with the key $\mathbf{ksk}^{\text{aut}}$, and the latter as *external key switching*, using the key $\mathbf{ksk}^{\text{ext}}$.

Automorphism Key Switching. Given a ciphertext $a = \sum_{i=0}^{N-1} a_i X^i$ encrypting a message m under the secret key $f'_Q(X) = f_Q(X^{-1})$, where the coefficient vector of f'_Q is given by $\phi(f'_Q) = (s_0, s_1, \dots, s_{N-1})^T = (f_{Q0}, -f_{Q_{N-1}}, \dots, -f_{Q1})^T$. The goal of *automorphism key switching* is to transform a into a new ciphertext b that encrypts the same message m under the secret key f_Q , where $\phi(f_Q) = (f_{Q0}, f_{Q1}, \dots, f_{Q_{N-1}})^T$. The automorphism key switching key is defined as:

$$\mathbf{ksk}^{\text{aut}} = \{\mathbf{ksk}_i^{\text{aut}} = (g + s_i) \cdot f_Q^{-1} \in R_Q \mid 0 \leq i < N\}$$

where $g \leftarrow \chi_\zeta^N$. We then generate a set of polynomials $P = \{P_i(X) = a \cdot X^i \mid 0 \leq i < N\}$. The output ciphertext b is computed as $b(X) = \sum_{i=0}^{N-1} \mathbf{ksk}_i^{\text{aut}} \cdot P_i(X)$, with the correction proof deferred to Appendix D.

External Key Switching. Let the output of the modulus switching be ACC' , which consists of n polynomials $\text{ACC}'_i = b_i = \sum_{j=0}^{N-1} b_{i,j} X^j \in \mathcal{R}_q$. The constant term of the i -th polynomial b_i encrypts the coefficient m_i of the target message polynomial $m = \sum_{i=0}^{n-1} m_i X^i$.

The external key switching procedure transforms these n scalar encryptions back into a single ciphertext $c = \text{NTRU}.\text{Enc}(m, f)$ under the base scheme. Following the principle of key switching, the external key switching key, denoted by $\mathbf{ksk}^{\text{ext}}$, encrypts the coefficients of the source key $\phi(f_Q) = (s_0, \dots, s_{N-1})^T$ under the target key f :

$$\mathbf{ksk}^{\text{ext}} = \{\mathbf{ksk}_i^{\text{ext}} = (g + s_i) \cdot f^{-1} \in R_q \mid 0 \leq i < N\}$$

Since the constant term of each ciphertext b_i can be extracted via the inner product $\langle \phi(b_i(X^{-1})), f_Q \rangle$, let $\tilde{\mathbf{b}}_i = \phi(b_i(X^{-1})) = (b_{i,0}, -b_{i,N-1}, \dots, -b_{i,1})^T$. We first construct a set of polynomials $P = \{P_i(X) = \sum_{j=0}^{n-1} \tilde{b}_{j,i} X^j \mid 0 \leq i < N\}$. Then the target ciphertext $c(X)$ is computed as $c(X) = \sum_{i=0}^{N-1} \mathbf{ksk}_i^{\text{ext}} \cdot P_i(X)$ and the correction proof is deferred to Appendix E. We further summarize the five types of auxiliary keys introduced in the proposed method in Table 1.

5 Security and Noise Analysis

5.1 Security Analysis

Under the decisional NTRU assumption, the proposed base and the accumulator schemes achieve IND-CPA security. As in standard FHE constructions, our bootstrapping procedure relies on a weak form of circular security, where the accumulator scheme is allowed to encrypt the secret key of the base scheme, and vice versa. This assumption is widely adopted in existing FHE schemes, including FINAL [2], and is necessary to enable bootstrapping.

For concrete security, we reuse the parameter sets from FINAL, which were validated against sublattice attacks using the analysis and scripts provided by Ducas and van Woerden. In particular, the selected parameters avoid the over-stretched NTRU regime and ensure that the block size β required by BKZ attacks exceeds the threshold corresponding to 128-bit classical security.

5.2 Noise Analysis

Let m be a ternary polynomial of degree at most $N-1$. For a scalar ciphertext in the accumulator scheme, the noise variance satisfies $\text{Var}(\text{err}(c)) \leq \text{Var}(g) + 4N\zeta^2$. Consider an external product $c_{\text{mult}} := c \boxtimes \mathbf{c}$, where $\mathbf{c}$ encrypts a monomial $v \in \mathbb{M}$. Assuming that the gadget decomposition $\mathbf{g}^{-1}(\cdot)$ is γ -subgaussian, the resulting noise variance is bounded by $\text{Var}(\text{err}(c_{\text{mult}})) \leq N \cdot l \cdot \gamma^2 \cdot \text{Var}(\text{err}(\mathbf{c})) + \text{Var}(\text{err}(c))$.

Extending this analysis to a sequence of k external products, denoted by $c' := c \boxtimes_{i=1}^k \mathbf{c}_i$, where all ciphertexts are freshly encrypted, we obtain

$$\text{Var}(\text{err}(c')) \leq N \cdot l \cdot \gamma^2 \cdot \sum_{i=1}^k \text{Var}(\mathbf{g}_i) + \text{Var}(\text{err}(c)) = (4 + (k+1) \cdot N \cdot l \cdot \gamma^2) \zeta^2$$

From Lemma 2, the sum of two ciphertexts must have noise magnitude strictly smaller than $(q-20)/8$. Consequently, each refreshed ciphertext must satisfy $\|\text{err}(\text{ct})\| < q/16 - 5/4$ to ensure correct decryption. This condition implies that choosing $q \in \tilde{O}(N)$ is sufficient. Since native bootstrapping applies the procedure independently to each polynomial coefficient, its noise behavior matches that of single-bit bootstrapping.

Following the analysis of Guimarães and Pereira, the noise growth of a single `MPMu1` operation with $\log B$ decomposition levels is approximately $\text{Var}(\text{err}(c_{\text{MPMu1}})) \approx \log B \cdot (N \cdot l \cdot \gamma^2 \cdot \zeta^2) + \text{Var}(\text{err}(c))$. For amortized bootstrapping using a ternary secret key with Hamming weight h , the noise contribution arises from h iterations of the main loop involving `MPMu1`, together with the overhead of handling ternary coefficients. The resulting noise variance can be bounded by

$$\begin{aligned} \text{Var}(\text{err}(c')) &\leq h \cdot (N \cdot l \cdot \gamma^2 \cdot \zeta^2) + (h+1) \log B \cdot (N \cdot l \cdot \gamma^2 \cdot \zeta^2) + \text{Var}(\text{err}(c)) \\ &\approx \mathcal{O}(N \cdot h \cdot \log B \cdot l \cdot \gamma^2 \cdot \zeta^2). \end{aligned}$$

Since the Hamming weight satisfies $h \ll N$, it can be treated as a small constant factor. As a result, selecting $q \in \tilde{O}(N)$ remains sufficient to guarantee correct decryption after amortized bootstrapping.

6 Evaluation

6.1 Experimental Setup

The goal of our evaluation is to validate the feasibility and scalability of the proposed amortized bootstrapping scheme, rather than to compete with highly optimized implementations. We therefore focus on relative performance trends and operation counts obtained from a proof-of-concept Python implementation.

All experiments were conducted on a workstation equipped with an Intel Core i9-14900K CPU (3.2 GHz) and 64 GB of RAM, running Ubuntu 22.04 LTS. The proposed framework was implemented and evaluated using Python 3.10. Unless otherwise specified, all evaluations are performed in a single-threaded setting without GPU acceleration.

6.2 Baseline: Native Bootstrapping

To establish a baseline for our evaluation, we first delineate the parameter selection process. Following the security analysis by Ducas and van Woerden regarding the NTRU fatigue point [6], FINAL identifies a parameter set that avoids the overstretched regime while maintaining a 128-bit security level. We adopt the same configuration in this work: we set the degree $n = 800$ and modulus $q = 131071 \approx 2^{17}$ for the base scheme, and $N = 1024$ with $Q = 912829 \approx 2^{19.8}$ for the accumulator scheme.

Recall that the native bootstrapping mechanism, as illustrated in Algorithm 1, reformulates the NTRU decryption circuit from a polynomial multiplication into n inner products. At the 128-bit security level, native bootstrapping requires exactly $N = 1024$ external products per message, corresponding to a single inner-product computation. This computational overhead is identical to that of FINAL.

6.3 Amortized Bootstrapping Performance

We next evaluate the performance gains achieved by amortized bootstrapping under different secret key Hamming weights h , and compare them against the native bootstrapping baseline. As sparse secret keys are known to be more vulnerable to dual hybrid attack, we follow the dual hybrid attack with meet-in-the-middle optimizations described in [4] and use the Lattice Estimator [1] to select secure parameters. To maintain a target security level of approximately 128 bits, we restrict the Hamming weight of the NTRU secret key to $h \geq 56$ in our base configuration.

Compared to the amortized bootstrapping results reported by Guimarães and Pereira, which leverage significantly larger ring dimensions (up to $n = 24576$), our evaluation focuses on a smaller parameter set with $n = 800$ to avoid the overstretched regime. Despite this conservative choice, our amortized bootstrapping achieves an algorithmic speedup of up to $1.8\times$ over native bootstrapping in terms of operation count, along with an approximately 33% reduction in bootstrapping key size. The results are summarized in Table 2.

Table 2. Evaluation of Amortized Bootstrapping (Amortized to Baseline)

h	Key Size (%)	Cost (%)	λ	Speedup
Baseline	100%	100%	-	-
56	67%	55%	128.2	1.8×
64	77%	63%	131.1	1.6×
72	86%	70%	133.4	1.4×

Speedup is measured in terms of operation count rather than wall-clock time.

6.4 Discussion

This work focuses on demonstrating the feasibility of an NTRU-based FHE scheme operating entirely over polynomial rings. Accordingly, the Python implementation is used to validate correctness and asymptotic operation counts rather than optimized runtime performance. Due to memory overhead and interpreter-level inefficiencies, wall-clock measurements do not fully reflect the theoretical advantages of amortized bootstrapping.

From an algorithmic perspective, amortized bootstrapping reduces the per-message operation cost by up to a factor of 1.8. Moreover, the impact on total execution time for amortized bootstrapping is limited, compared Guimarães and Pereira, since our experiments are based on relatively small ring dimension n .

7 Conclusion

Fully homomorphic encryption (FHE) is a fundamental building block for privacy-preserving machine learning (PPML). Motivated by the comprehensive security analysis of NTRU by Ducas and van Woerden, Bonte *et al.* demonstrated that it is feasible to construct an FHE scheme under the NTRU assumption while benefiting from compact ciphertexts. In this work, we revisit this line of research by reformulating the matrix-based NTRU construction into a standard polynomial-ring setting, thereby restoring the algebraic structure required for amortized bootstrapping.

By leveraging sparse secret keys and monomial-by-polynomial multiplication, we adapt a recent amortized bootstrapping technique to the NTRU setting. The resulting scheme combines compact ciphertexts with efficient amortized bootstrapping, reducing the asymptotic computational cost and the size of the bootstrapping key while maintaining conservative security parameters controlled by the Hamming weight of the secret key.

As future work, we plan to develop optimized implementations in low-level languages and explore parallel execution through multi-threading or GPU acceleration. From an algorithmic perspective, since NTRU naturally supports integer plaintexts, extending our construction from binary polynomials to integer polynomials remains an essential direction toward practical deployment.

References

1. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. Cryptology ePrint Archive, Paper 2015/046 (2015), <https://eprint.iacr.org/2015/046>
2. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: Final: Faster fhe instantiated with ntru and lwe. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022*. pp. 188–215. Springer Nature Switzerland, Cham (2022)
3. Bos, J.W., Lauter, K., Loftus, J., Naehrig, M.: Improved security for a ring-based fully homomorphic encryption scheme. In: Stam, M. (ed.) *Cryptography and Coding*. pp. 45–64. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
4. Cheon, J.H., Hhan, M., Hong, S., Son, Y.: A hybrid of dual and meet-in-the-middle attack on sparse and ternary secret LWE. Cryptology ePrint Archive, Paper 2019/1114 (2019), <https://eprint.iacr.org/2019/1114>
5. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. Cryptology ePrint Archive, Paper 2016/870 (2016), <https://eprint.iacr.org/2016/870>
6. Ducas, L., van Woerden, W.: Ntru fatigue: How stretched is overstretched? In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021*. pp. 3–32. Springer International Publishing, Cham (2021)
7. Ducas, L., Micciancio, D.: Fhew: Bootstrapping homomorphic encryption in less than a second. Cryptology ePrint Archive, Paper 2014/816 (2014), <https://eprint.iacr.org/2014/816>, <https://eprint.iacr.org/2014/816>
8. Guimarães, A., Pereira, H.V.L.: Fast amortized bootstrapping with small keys and polynomial noise overhead. In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. p. 2967–2981. CCS ’25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3719027.3765181>, <https://doi.org/10.1145/3719027.3765181>
9. Hoffstein, J., Pipher, J., Silverman, J.H.: Ntru: A ring-based public key cryptosystem. In: Buhler, J.P. (ed.) *Algorithmic Number Theory*. pp. 267–288. Springer Berlin Heidelberg, Berlin, Heidelberg (1998)
10. Kuru, K., Kuru, K.: Blockchain-enabled privacy-preserving machine learning authentication with immersive devices for urban metaverse cyberspaces. In: *2024 20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*. pp. 1–8 (2024). <https://doi.org/10.1109/MESA61532.2024.10704877>
11. López-Alt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: *Proceedings of the Forty-Fourth Annual ACM Symposium on Theory of Computing*. p. 1219–1234. STOC ’12, Association for Computing Machinery, New York, NY, USA (2012). <https://doi.org/10.1145/2213977.2214086>
12. Ma, J., Zheng, Y., Feng, J., Zhao, D., Wu, H., Fang, W., Tan, J., Yu, C., Zhang, B., Wang, L.: SecretFlow-SPU: A performant and User-Friendly framework for Privacy-Preserving machine learning. In: *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. pp. 17–33. USENIX Association, Boston, MA (Jul 2023), <https://www.usenix.org/conference/atc23/presentation/ma>
13. Micciancio, D., Sorrell, J.: Ring packing and amortized fhew bootstrapping. In: *International Colloquium on Automata, Languages and Programming* (2018), <https://api.semanticscholar.org/CorpusID:47002757>

14. OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al.: Gpt-4 technical report (2024), <https://arxiv.org/abs/2303.08774>
15. Team, G., Anil, R., Borgeaud, S., Alayrac, J.B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A.M., Hauth, A., et al.: Gemini: A family of highly capable multimodal models (2025), <https://arxiv.org/abs/2312.11805>

A Proof of Lemma 1

Proof. To recover the message, we first compute the phase function $r = \varphi(c)$:

$$\begin{aligned}
 r &= c \cdot f \pmod{q} = ((g + \Delta \cdot m) \cdot f^{-1}) \cdot f \pmod{q} \\
 &= (g + \Delta \cdot m) \cdot (f^{-1} \cdot f) \pmod{q} \\
 &= g + \Delta \cdot m - u \cdot q, \text{ for some } u \in \mathbb{Z}
 \end{aligned}$$

The decryption process scales r and rounds the result:

$$\begin{aligned}
 m' &= \left\lfloor \frac{2r}{q} \right\rfloor \pmod{2} \\
 &= \left\lfloor \frac{2(g + \Delta \cdot m - u \cdot q)}{q} \right\rfloor \pmod{2} \\
 &= \left\lfloor \frac{2g}{q} + \frac{2m}{q} \left(\frac{q}{2} + \epsilon \right) - 2u \right\rfloor \pmod{2} \\
 &= \left\lfloor \frac{2g}{q} + \frac{2\epsilon m}{q} \right\rfloor + m - 2u \pmod{2}
 \end{aligned}$$

Decryption is correct ($m' = m$) if and only if the rounding term of each index i that $|\frac{2g_i + 2\epsilon m_i}{q}| < 1/2$ holds. Given that g is ternary ($|g_i| \leq 1$), m is binary ($|m_i| \leq 1$), and ϵ is the rounding error with $|\epsilon| \leq 1/2$, the absolute value is bounded by:

$$\left| \frac{2g_i + 2\epsilon m_i}{q} \right| \leq \frac{2(1) + 2(1/2)(1)}{q} = \frac{3}{q} < \frac{1}{2}$$

Thus, for any $q > 6$, the condition holds for all indices i , ensuring $m' = m$. $\square$

B Proof of Lemma 2

Similar to the techniques in [7,2], consider:

$$\begin{aligned}
 &c_{\text{NAND}} \cdot f - \frac{q}{2} \cdot (1 - a \circ b) \\
 &= (\text{evk} - c_0 - c_1) \cdot f - \frac{q}{2} (1 - a \circ b) \\
 &= (g + \lfloor \frac{5 \cdot q}{8} \rfloor) - (g_0 + \lfloor \frac{q}{4} \rfloor \cdot a) - (g_1 + \lfloor \frac{q}{4} \rfloor \cdot b) - \frac{q}{2} (1 - a \circ b) \\
 &= g - g_0 - g_1 + \frac{5q}{8} + \epsilon - \frac{q}{4} \cdot a - \epsilon_0 \cdot a - \frac{q}{4} \cdot b - \epsilon_1 \cdot b - \frac{q}{2} + \frac{q}{2} (a \circ b) \\
 &= g - g_0 - g_1 + \frac{q}{8} - \frac{q}{4} (a - b)^{\circ 2} + e
 \end{aligned}$$

where $e = \epsilon - \epsilon_0 a - \epsilon_1 b$. Since $\epsilon, \epsilon_0, \epsilon_1$ are rounding errors, we have $\max(|\epsilon|, |\epsilon_0|, |\epsilon_1|) \leq 1/2$. Then the magnitude of the error term satisfies $|e| < 3/2$.

Proof. Let $y = 1 - a \circ b$. Based on the noise analysis above, we have $r = c_{\text{NAND}} \cdot f = (g - g_0 - g_1 + \delta + e) + \frac{q}{2}y$, where $\delta_i = q/8$ if $a_i = b_i$, and $\delta_i = -q/8$ if $a_i \neq b_i$. The decryption process computes:

$$\begin{aligned} y' &= \left\lfloor \frac{2r}{q} \right\rfloor \pmod{2} \\ &= \left\lfloor \frac{2(g - g_0 - g_1 + e + \delta)}{q} + y \right\rfloor \pmod{2} \end{aligned}$$

Decryption is successful ($y'_i = y_i$) if the noise term satisfies:

$$\left| \frac{2(\phi(g)_i - \phi(g_0)_i - \phi(g_1)_i + e) + 2\delta_i}{q} \right| < \frac{1}{2}$$

Substituting $\delta_i = \pm q/8$, this condition is equivalent to:

$$\left| \frac{2(\phi(g)_i - \phi(g_0)_i - \phi(g_1)_i + e)}{q} \pm \frac{1}{4} \right| < \frac{1}{2}$$

which implies $\|\phi(g)_i - \phi(g_0)_i - \phi(g_1)_i\| < (\frac{1}{2} - \frac{1}{4})\frac{q}{2} - |e| = \frac{q}{8} - \frac{3}{2} = \frac{q-12}{8}$. Given $|\phi(g)_i| \leq 1$ as g is the fresh noise from the evaluation key, the result holds if:

$$|\phi(g_0)_i + \phi(g_1)_i| < \frac{q-12}{8} - 1 = \frac{q-20}{8}, \text{ for all } 0 \leq i < n$$

C Correctness of the External Product

Recall that the scalar ciphertext is $c = g/f_Q + \Delta \cdot u$ and the vector encryption is $\mathbf{c} = \mathbf{g}/f_Q + \mathbf{g} \cdot v$. Based on the property of the gadget decomposition where $\mathbf{g}^{-1}(c) \cdot \mathbf{g} = c$, the expansion of the product $c_{\text{mult}} = c \boxtimes \mathbf{c}$ is derived as follows:

$$\begin{aligned} c_{\text{mult}} &= \mathbf{g}^{-1}(c) \cdot \mathbf{c} \\ &= \mathbf{g}^{-1}(c) \cdot (\mathbf{g}/f_Q + \mathbf{g} \cdot v) \\ &= (\mathbf{g}^{-1}(c) \cdot \mathbf{g})/f_Q + (\mathbf{g}^{-1}(c) \cdot \mathbf{g} \cdot v) \\ &= (\mathbf{g}^{-1}(c) \cdot \mathbf{g})/f_Q + (c \cdot v) \\ &= (\mathbf{g}^{-1}(c) \cdot \mathbf{g})/f_Q + (g/f_Q + \Delta \cdot u) \cdot v \\ &= (\mathbf{g}^{-1}(c) \cdot \mathbf{g} + g \cdot v)/f_Q + \Delta \cdot u \cdot v \end{aligned}$$

Let $g_{\text{mult}} = \mathbf{g}^{-1}(c) \cdot \mathbf{g} + g \cdot v$. Then, $c_{\text{mult}} = g_{\text{mult}}/f_Q + \Delta \cdot u \cdot v$ is a valid scalar encryption of $u \cdot v$, provided that g_{mult} is sufficiently small.

D Proof of Automorphism Key Switching in Section 4.4

Proof. Let $b(X) = \sum_{i=0}^{N-1} \mathbf{k} \mathbf{s}_i^{\text{aut}} \cdot P_i(X)$. Decrypting $b(X)$ with the secret key f_Q yields:

$$\begin{aligned}
r(X) &= b(X) \cdot f_Q = \sum_{i=0}^{N-1} (g + s_i) \cdot f_Q^{-1} \cdot P_i(X) \cdot f_Q \\
&= \sum_{i=0}^{N-1} (g + s_i) \cdot P_i(X) \approx \sum_{i=0}^{N-1} s_i \cdot P_i(X) \\
&= \sum_{i=0}^{N-1} s_i \cdot \left(\sum_{j=0}^{N-1} a_j X^j \right) \cdot X^i = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} s_i a_j X^{i+j} \\
&= \sum_{i=0}^{N-1} \left(\sum_{j=0}^i a_{i-j} \cdot s_i - \sum_{j=i+1}^{N-1} a_{N+i-j} \cdot s_i \right) \cdot X^i = a * f_Q
\end{aligned}$$

This confirms that $b(X)$ correctly decrypts to m under key f_Q .

E Proof of External Key Switching in Section 4.4

Proof. Decrypting $c(X)$ with the secret key f yields:

$$\begin{aligned}
r(X) &= c(X) \cdot f = \sum_{i=0}^{N-1} (g + s_i) \cdot f^{-1} \cdot P_i(X) \cdot f \\
&= \sum_{i=0}^{N-1} (g + s_i) \cdot (f^{-1} \cdot f) \cdot P_i(X) \\
&= \sum_{i=0}^{N-1} (g + s_i) \cdot P_i(X) \approx \sum_{i=0}^{N-1} s_i \cdot P_i(X) \\
&= \sum_{i=0}^{N-1} s_i \cdot \sum_{j=0}^{n-1} \tilde{b}_{j,i} \cdot X^j = \sum_{i=0}^{n-1} \left(\sum_{j=0}^{N-1} s_j \cdot \tilde{b}_{i,j} \right) \cdot X^i \\
&= \sum_{i=0}^{n-1} \langle \phi(b_i(X^{-1})), f_Q \rangle \cdot X^i = \sum_{i=0}^{n-1} m_i \cdot X^i
\end{aligned}$$

Confirms that decrypting $c(X)$ with key f can get the expected polynomial m .